

Learn More

검색어를 입력하세요.

Q

≡

🏠 (/)

📖 점프 투 자바

(/book/31)

00장 들어가기 전에

00-01 머리말

00-02 주요변경이력

00-03 저자소개

00-04 책 구입 안내

01장 자바란 무엇인가?

01-01 자바란?

01-02 자바로 무엇을 할 수 있을까?

01-03 자바 둘러보기

01-04 자바의 8가지 특징

02장 자바 시작하기

02-01 자바 코드의 구조 살펴보기

02-02 변수와 자료형

02-03 이름 짓는 규칙
02-04 주석이란?
03장 자바의 기초 - 자료형
03-01 숫자
03-02 불
03-03 문자
03-04 문자열
03-05 StringBuffer
03-06 배열
03-07 리스트
03-08 맵
03-09 집합
03-10 상수 집합
03-11 형 변환과 final
03장 되새김 문제
04장 제어문 이해하기
04-01 if 문
04-02 switch-case 문
04-03 while 문
04-04 for 문
04-05 for each 문
04장 되새김 문제

05장 객체 지향 프로그래밍
05-01 객체 지향 프로그래밍이란?
05-02 클래스
05-03 메서드 더 살펴보기
05-04 값에 의한 호출과 객체에 의한 호출
05-05 상속
05-06 생성자
05-07 인터페이스
05-08 다형성
05-09 추상 클래스
05장 되새김 문제
06장 자바의 입출력
06-01 콘솔 입출력
06-02 파일 입출력
06장 되새김 문제
07장 자바 날개 달기
07-01 패키지
07-02 접근 제어자
07-03 스택틱
07-04 예외 처리
07-05 스레드

07-06 함수형 프로그래밍
07장 되새김 문제
08장 자바 프로그래밍, 어떻게 시작해야 할까?
08-01 내가 프로그램을 만들 수 있을까?
08-02 3과 5의 배수 합하기
08-03 게시판 페이지징하기
08-04 자릿수 구하기
08-05 공백을 제외한 글자 수 세기
09장 자바 코딩 면허 시험 15제
10장 정답 및 풀이
11장 부록
11-01 챗GPT와 함께 자바 공부하기
11-02 public 클래스
12장 마치며



05-02 클래스

↑



광고가 출력될 위치입니다.

광고가 출력될 위치입니다.

이 책에서는 클래스에 대한 개념적인 설명을 하기보다는 가장 간단한 클래스를 작성하는 것에서부터 시작하여 조금씩 클래스의 규모를 키워 가며 설명할 것이다.

클래스와 객체

객체 변수란?

메서드란?

객체 변수는 공유되지 않는다

클래스와 객체

객체가 무엇인지를 알기 위해 먼저 `Animal` 클래스를 다음과 같이 `Sample.java` 파일에 작성하자.

```
class Animal {  
  
  
    public class Sample {  
        public static void main(String[] args) {  
        }  
    }  
}
```

Copy

점프 투 자바

꼭 읽고 넘어가기

`Animal` 클래스를 `Sample.java` 파일에 작성한 것은 책의 원활한 설명을 위해서이다. 보통 클래스는 특별한 경우가 아니라면 파일 단위로 하나씩 작성한다. 그러므로 `Animal` 클래스는 `Animal.java` 파일에 단독으로 작성하는 것이 일반적인 방법이다. 하지만 이 책의 예제 대부분은 `Sample.java` 파일만 작성하여 실행할 수 있도록 구성하였으니 참고하기 바란다.

`Animal` 클래스는 가장 간단한 형태의 클래스이다. 클래스의 선언만 있고 내용이 없는 껍데기뿐인 클래스이다. 하지만 이런 껍데기뿐인 클래스도 아주 중요한 기능을 가지고 있다. 바로 **객체(object)**를 만드는 기능이다. 객체는 다음과 같이 만들 수 있다.

```

class Animal {
}

public class Sample {
    public static void main(String[] args) {
        Animal cat = new Animal();
    }
}

```

`new` 는 객체를 생성할 때 사용하는 키워드이다. 이렇게 하면 `Animal` 클래스의 인스턴스(instance)인 `cat`, 즉 `Animal`의 객체가 만들어진다.

점프 투 자바

객체와 인스턴스의 차이는 무엇일까?

클래스에 의해서 만들어진 객체를 인스턴스라고도 한다. 그렇다면 객체와 인스턴스의 차이는 무엇일까?
`Animal cat = new Animal()` 이렇게 만들어진 `cat`은 객체이다. 그리고 `cat`이라는 객체는 `Animal`의 인스턴스이다. 인스턴스라는 말은 특정 객체(여기서는 `cat`)가 어떤 클래스(여기서는 `Animal`)의 객체 인지 를 관계 위주로 설명할 때 사용된다. 즉, ‘`cat`은 인스턴스’보다는 ‘`cat`은 객체’라는 표현이, ‘`cat`은 `Animal`의 객체’보다는 ‘`cat`은 `Animal`의 인스턴스’라는 표현이 훨씬 잘 어울린다.

다음 그림을 통해 클래스를 이해해 보자. 과자 모양을 찍어 내는 과자 틀은 클래스에 비유할 수 있고, 과자 틀에 의해 만들어진 과자들은 객체에 비유할 수 있다.



다음과 같이 무수히 많은 동물 객체(여기서는 `cat`, `dog`, `horse`, ...)들을 `Animal` 클래스로 만들 수 있다.

```

Animal cat = new Animal();
Animal dog = new Animal();
Animal horse = new Animal();
...

```

여기서 설명한 내용이 이해되었다면 클래스는 70% 이상 알았다 할 수 있다. 클래스는 결코 어렵지 않다.

객체 변수란?

Animal이라는 ‘껍데기’ 클래스를 조금 더 발전시켜 보자. Animal 클래스에 의해 만들어진 동물 객체의 이름을 지어 보자.

```
class Animal {
    String name;
}

public class Sample {
    public static void main(String[] args) {
        Animal cat = new Animal();
    }
}
```

Animal 클래스에 name이라는 String 변수를 추가했다. 이렇게 클래스에 선언된 변수를 **객체 변수(instance variable)**라고 한다. 클래스에 의해 생성되는 것은 객체, 그리고 그 클래스에 선언된 변수는 객체 변수라고 생각하면 쉽다.

객체 변수는 인스턴스 변수, 멤버 변수, 속성이라고도 한다.

객체 변수를 만들었으니 이제 객체 변수를 사용해 보자. 먼저 객체 변수 또한 변수이므로 값을 대입할 수 있다. 대입하기 전에 객체 변수는 현재 어떤 값을 가지고 있는지를 출력해 보자. 객체 변수를 출력하려면 먼저 객체 변수에 어떻게 접근해야 하는지를 알아야 한다. 객체 변수는 다음과 같이 도트 연산자(.)를 이용하여 접근할 수 있다.

객체, 객체변수

Animal cat = new Animal() 처럼 cat이라는 객체를 생성했다면 이 cat 객체의 객체 변수 name에는 다음과 같이 접근할 수 있다.

```
cat.name    // 객체: cat, 객체변수: name
```

객체 변수에 어떤 값이 대입되어 있는지 다음과 같이 작성하여 출력해 보자.

기말고사 핵심 문제 무료 배:

학년별, 출판사별 맞춤형
기말고사 예상 문제 무료 배포 중!
지금 바로 확인해 보세요.



```
class Animal {  
    String name;  
}  
  
public class Sample {  
    public static void main(String[] args) {  
        Animal cat = new Animal();  
        System.out.println(cat.name);  
    }  
}
```

```
null
```

`cat.name` 을 출력한 결과값으로 `null`이 나왔다. `null`은 값이 할당되어 있지 않은 상태를 말한다. 객체 변수로 `name`을 선언했지만 아무런 값도 대입하지 않았기 때문에 `null`이라는 값이 출력된 것이다.

메서드란?

앞서 객체 변수에 접근하는 방법에 대해서 알아보았다. 이번에는 객체 변수에 값을 대입하는 방법에 대해서 알아보자. 객체 변수에 값을 대입하는 방법에는 여러 가지가 있지만 여기서는 메서드를 이용하는 방법에 대해서 알아보려고 한다. 클래스에는 객체 변수와 함께 메서드가 있다. 메서드(method)는 클래스 내에 구현된 함수를 말한다.

이제 메서드를 이용하여 `Animal` 클래스의 객체 변수인 `name`에 값을 대입해 보자. 다음과 같이 `setName` 메서드를 추가해 보자.

```
class Animal {
    String name;

    public void setName(String name) {
        this.name = name;
    }
}

public class Sample {
    public static void main(String[] args) {
        Animal cat = new Animal();
        System.out.println(cat.name);
    }
}
```

`Animal` 클래스에 추가된 `setName` 메서드는 다음과 같은 형태의 메서드이다.

- 입력: `String name`
- 출력: `void` ('반환값 없음'을 의미)

즉, 입력으로 `name`이라는 문자열을 받고 출력은 없는 형태의 메서드이다.

메서드의 입출력에 대한 자세한 내용은 05-3절에 준비되어 있다. 여기서 메서드가 도무지 이해가 되지 않는다면 05-3절을 먼저 읽고 와도 좋다.

이번에는 `setName` 메서드의 내부를 살펴보자. `setName` 메서드는 다음의 문장을 가지고 있다.

```
this.name = name;
```

이 문장에서 `this`에 대해서 이해하는 것은 꽤 중요하다. 하지만 이 문장에 대한 설명은 잠시 보류하고 우선 이 메서드를 호출하는 방법을 먼저 알아보자. 객체 변수에 접근하기 위해서 도트 연산자(`.`)로 접근할 수 있었던 것과 마찬가지로 객체가 메서드를 호출하기 위해서는 객체.메서드 로 호출해야 한다. 즉, 우리가 만든 `setName` 메서드를 호출하려면 다음과 같이 호출해야 한다.

```
cat.setName("boby");
```

이때 setName 메서드는 입력 항목으로 "boby"와 같이 문자열을 전달해야 한다. setName 메서드를 호출할 수 있도록 main 메서드를 다음과 같이 수정해 보자.

```
class Animal {
    String name;

    public void setName(String name) {
        this.name = name;
    }
}

public class Sample {
    public static void main(String[] args) {
        Animal cat = new Animal();
        cat.setName("boby"); // 메서드 호출
        System.out.println(cat.name);
    }
}
```

이렇게 수정하면 cat.name을 출력하기 전에 setName 메서드가 먼저 호출된다.

아까 설명을 잠시 보류한 setName 메서드의 다음 문장을 다시 살펴보자.

```
this.name = name;
```

main 메서드에서 cat.setName("boby") 는 "boby"를 입력값으로 하여 setName 메서드를 호출했기 때문에 setName 메서드의 입력 항목 name에는 "boby"라는 문자열이 전달될 것이다.

따라서 setName 메서드의 this.name = name; 문장은 다음과 같은 뜻이기도 하다.

```
this.name = "boby";
```

그리고 setName 메서드의 this는 Animal 클래스에 의해서 생성된 객체를 지칭한다. Animal cat = new Animal() 과 같이 cat이라는 객체를 만들고, cat.setName("boby") 와 같이 cat 객체를 사용하여 setName 메서드를 호출하면 setName 메서드 내부에 선언된 this는 바로 cat 객체를 지칭한다.

만약 `Animal dog = new Animal()` 로 `dog` 객체를 만든 후 `dog.setName("happy")` 와 같이 호출한다면 `setName` 메서드 내부에 선언된 `this`는 바로 `dog` 객체를 가리킨다.

따라서 `this.name = "boby"`; 문장은 다시 다음과 같이 해석된다.

```
cat.name = "boby";
```

`cat.name` 과 같이 하면 객체 변수에 접근할 수 있음을 우리는 알고 있다. 객체 변수에 값을 대입하는 방법도 아주 쉽다. 그냥 변수에 값을 대입하는 것과 마찬가지로 방법이다.

```
객체.객체변수 = 값
```

따라서 `cat.name = "boby"` 라는 문장은 객체 `cat`의 객체 변수 `name`에 `boby`라는 값을 대입한다.



**이 게임은 매우 사실적이어서 !
가치가 있습니다**

game-raiders

이제 `main` 메서드를 다시 실행해 보자. 다음과 같은 문자열이 출력되는 것을 확인할 수 있을 것이다.

```
boby
```

`cat.name` 은 이제 `null`이 아니라 `boby`임을 확인할 수 있다.

객체 변수는 공유되지 않는다

`main` 메서드를 다음과 같이 변경해 보자.

```

class Animal {
    String name;

    public void setName(String name) {
        this.name = name;
    }
}

public class Sample {
    public static void main(String[] args) {
        Animal cat = new Animal();
        cat.setName("boby");

        Animal dog = new Animal();
        dog.setName("happy");
    }
}

```

cat 객체에는 "boby"라는 이름을 대입하고, dog 객체에는 "happy"라는 이름을 대입했다. 이렇게 하면 setName 메서드에 의해 다음과 같은 문장이 두 번 실행될 것이다.

```

cat.name = "boby";
dog.name = "happy";

```

그런데 이럴 경우 dog.name = "happy" 라는 문장이 나중에 수행되므로 cat.name의 값도 "happy"라는 값으로 변경되지 않을까? Animal 클래스의 객체 변수 name이 cat 객체와 dog 객체 간 서로 공유되는 변수라면 아마도 그럴 것이다. 다음과 같이 작성하여 확인해 보자.

```

class Animal {
    String name;

    public void setName(String name) {
        this.name = name;
    }
}

public class Sample {
    public static void main(String[] args) {
        Animal cat = new Animal();
        cat.setName("boby"); // 메서드 호출

        Animal dog = new Animal();
        dog.setName("happy");

        System.out.println(cat.name);
        System.out.println(dog.name);
    }
}

```

```

boby
happy

```

그런데 결과를 보면 name 객체 변수는 공유되지 않는다는 것을 확인할 수 있다. 이 부분은 정말 너무너무 중요해서 강조하고 또 강조해도 지나치지 않다. 클래스에서 가장 중요한 부분은 **객체 변수의 값이 독립적으로 유지된다는 점**이다. 사실 이 점이 바로 클래스 존재의 이유이기도 하다.

객체 지향적(object oriented)이라는 말의 의미도 곱씹어 보면 결국 이 객체 변수의 값이 독립적으로 유지되기 때문에 가능한 것이다.

객체 변수의 값은 공유되지 않지만 07-3절에서 공부할 static을 이용하게 되면 객체 변수를 공유하도록 만들 수도 있다.

마지막 편집일시 : 2025년 8월 20일 4:28 오후

광고가 출력될 위치입니다.

광고가 출력될 위치입니다.

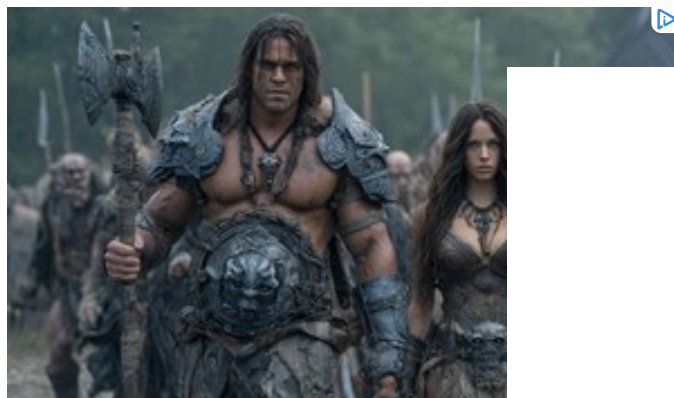
뉴그랜드 정수기 프리스탠딩..

열기

댓글 5

피드백

- 이전글 : 05-01 객체 지향 프로그래밍이란?
- 다음글 : 05-03 메서드 더 살펴보기



컴퓨터가 있는 사람이라면 꼭 해
임!

RAID: Shadow Legends